

Linear Models and Text Classification
NLP Classification Pipeline: Data Acquisition → Text Cleaning → Pre-processing → Feature Extraction → Machine Learning → Evaluation → Deployment

- ◆ **Preprocessing:** Goal: convert text into **standardized form for modeling**; Techniques: **tokenization**, removing spaces/noise, **punctuation splitting**, **lowercasing**, **stopword removal**, **stemming/lemmatization**; Key principle: **standardization improves generalization but loses nuanced information**
- ◆ **Naïve Bayes (Generative Model):** Models **joint probability** $p(\mathbf{x}, \mathbf{y})$ by learning: $p(\mathbf{y})$: prior probability of class; $p(\mathbf{x}|\mathbf{y})$: likelihood of document given class **Prior:** $\pi_j = \text{count}(\mathbf{y} = j) / n$ **Likelihood (Word Distribution):** $\tau_{wj} = \text{count}(\text{word } w \text{ in class } j) / \text{total words in class } j$
- ◆ **Naïve Bayes Assumption:** Words are **conditionally independent given label**, meaning documents are modeled as sampling words independently after choosing class
- ◆ **Laplace Smoothing:** Prevents zero probabilities using: $\tau_{wj} = (\text{count}(w, j) + \lambda) / (\text{count}(j) + |\mathbf{V}| \cdot \lambda)$; Fixes issue where unseen words make probability zero
- ◆ **Prediction:** Compute $p(\mathbf{x}, \mathbf{y}) = p(\mathbf{y}) \times \prod p(\text{word} | \mathbf{y})$ and choose label with highest value
- ◆ **Numerical Underflow:** Multiplying many small probabilities → zero; Solution: compute in **log space (sum of logs instead of product)**
- ◆ **Pros / Cons:** Pros: **simple, fast, works well on small data**; Cons: independence assumption unrealistic, limited expressiveness
- ◆ **Logistic Regression (Discriminative Model):** Directly models $p(\mathbf{y} | \mathbf{x})$ instead of joint probability; Learns parameters **w (weights)** and **b (bias)** to separate classes
- ◆ **Linear Model:** Score: $s = \mathbf{w}^T \Phi(\mathbf{x}) + b$ Prediction: $y = +1$ if score > 0 , else -1
- ◆ **Sigmoid Function:** $\sigma(z) = 1 / (1 + e^{-(z)})$; Converts score to **probability between 0 and 1**
- ◆ **Objective Function:** Minimize loss: $L(\mathbf{w}, \mathbf{b}) = \sum -\log \sigma(\mathbf{y}^T (\mathbf{w}^T \Phi(\mathbf{x}) + b))$; Equivalent to maximizing probability of correct labels
- ◆ **Optimization:** Use **gradient descent** to update parameters
- ◆ **Convexity:** Loss function is **convex (bowl-shaped)**, so gradient descent finds global optimum
- ◆ **Pros / Cons:** Pros: **higher accuracy on large datasets, flexible features**; Cons: requires optimization and more data
- ◆ **Feature Engineering: Unigrams (Bag of Words):** Feature: **count of each word in vocabulary**; Pros: simple, efficient, interpretable; Cons: ignores **word order and context**
- ◆ **Bigrams:** Feature: **two-word sequences**; Captures short context (e.g., “not good”); Trade-off: **more parameters (up to $|\mathbf{V}|^2$)** and risk of **overfitting**; Use **unigrams + bigrams together** for better performance

Word Vectors (Word2Vec)
Distributional Hypothesis: Words appearing in similar contexts have similar meanings, summarized as “You shall know a word by the company it keeps”
Goal of Word Vectors: Learn dense, low-dimensional vectors for each word that capture lexical semantics such as synonymy, antonymy, hypernymy, similarity, and other properties like sentiment or formality

- ◆ **Count-Based Word Vectors: Co-occurrence Counting:** Build vectors by **counting neighboring words within a sliding window**, forming a term-term matrix where each row is a word vector
- ◆ **Problem:** Produces **high-dimensional, sparse vectors with many zeros**, which are inefficient and do not generalize well
- ◆ **Cosine Similarity:** Measures similarity using: $\cos(\theta) = (\mathbf{u} \cdot \mathbf{v}) / (||\mathbf{u}|| \cdot ||\mathbf{v}||)$; Interpretation: ≈ 1 : very similar ≈ 0 : unrelated ≈ -1 : opposite meaning
- ◆ **Word2Vec (Dense Embeddings):** Learn word vectors via a “fake” supervised task predicting context words, instead of directly using counts
- ◆ **Dataset Construction:** Use sliding window: positive pairs: (**center word, context word**); negative pairs: **randomly sampled words**
- ◆ **Negative Sampling:** Sample negatives using **frequency ^{α}** ($\alpha \approx 0.75$) to reduce over-sampling of common words like “the”
- ◆ **Model Parameters:** Learn: word vectors **$\mathbf{v}_w$ (final embeddings)**; context vectors **$\mathbf{c}_w$ (helper parameters, discarded later)**
- ◆ **Objective:** Maximize similarity for real pairs and minimize for fake pairs, using a **logistic regression-like objective optimized with gradient descent**
- ◆ **Optimization:** Both word and context vectors are learned simultaneously, making the objective **non-convex**, but gradient descent works well in practice
- ◆ **Output:** Final embeddings are **word vectors $\mathbf{v}_w$** , while context vectors are discarded
- ◆ **Properties of Word Vectors: Semantic Similarity:** Similar words have similar vector directions, capturing relationships like synonymy and semantic similarity
- ◆ **Vector Arithmetic (Analogies):** Relationships encoded as linear offsets: $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$; General form: $\mathbf{v}_X \approx \mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$
- ◆ **Analogy Solving:** Find word whose vector is **closest (cosine similarity)** to computed vector expression
- ◆ **Mean Pooling:** Represent document by **averaging word vectors**, producing a fixed-length vector for classification Limitation: treats text as **bag-of-words**, ignoring order and structure
- ◆ **Benefits:** Enable **generalization to rare/unseen words**, since similar words share nearby vectors; Often **pretrained on large corpora** (Wikipedia, news, books) and reused for downstream tasks
- ◆ **Limitations: Sparsity (Count Methods):** Co-occurrence matrices are **large and sparse**, making them inefficient; Bias: Word vectors encode **historical and societal biases**, e.g., gender stereotypes in occupations; **Polysemy:** One vector per word cannot distinguish **multiple meanings of same word** (e.g., “bank”); **Word Order:** Methods like mean pooling ignore **word order and syntax**, limiting understanding

Neural Networks

- ◆ **Neural Networks:** Neural networks are **learnable non-linear functions**, unlike linear models which can only represent linear relationships; Without non-linearity, stacked layers collapse into a **single linear function**, so depth adds no power
- ◆ **Importance of Non-Linearity:** Activation functions enable **learning complex, non-linearly separable patterns** (e.g., XOR); Without them, network behaves like a linear classifier

- ◆ **Activation Functions: Sigmoid:** maps values to (0,1), used for probabilities; **Tanh:** outputs centered around zero, often preferred over sigmoid; **ReLU:** $\max(0, z)$, fast and widely used
- ◆ **Neural Network Structure: Hidden Layer:** Acts as **feature extractor**, where each neuron behaves like a small linear classifier producing learned features
- ◆ **Final Layer: Binary classification:** logistic regression layer; **Multi-class:** linear + softmax layer; Only final layer changes when adapting tasks
- ◆ **Matrix Form:** $\mathbf{W}: h \times d$ matrix, $\mathbf{b}$: length h vector; Parameters: $\Theta = (\mathbf{W}, \mathbf{b}, \mathbf{v}, \mathbf{c})$
- ◆ **Depth vs Width: Deeper networks (more layers)** usually outperform wider ones because they learn more complex hierarchical features
- ◆ **Neural Networks as Feature Learners:** Each hidden unit computes a **new feature as a linear classifier over inputs**, allowing automatic feature construction (e.g., learning negation instead of manually adding bigrams)
- ◆ **Deep Learning as Building Blocks: Linear Block:** $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, with parameters $\mathbf{W}$ ($d_{\text{out}} \times d_{\text{in}}$) and $\mathbf{b}$; **Non-linearity Block:** applies element-wise activation functions like sigmoid, tanh, or ReLU with no parameters; **Word Vector Block:** maps each word to a vector, with parameters consisting of one vector per word (total $|\mathbf{V}| \times d$); **Mean Pooling Block:** averages word vectors across sequence length to produce a fixed-size representation; **Architecture:** neural networks are formed by stacking these blocks to build flexible models
- ◆ **Deep Averaging Network (DAN):** Uses pipeline: word vectors → mean pooling → MLP, and often outperforms Naïve Bayes on sentiment tasks
- ◆ **CNN for Text:** Applies neural networks over fixed-size word windows (n-grams) and pools outputs to create representations, capturing local patterns and generalizing to unseen n-grams via embeddings
- ◆ **Training Neural Networks**
- ◆ **Backpropagation:** computes gradients using chain rule through all layers for efficient training
- ◆ **Stochastic Gradient Descent (SGD):** uses mini-batches to approximate gradients, making training scalable, where one epoch is a full pass over the dataset
- ◆ **Batch Size:** large batches give more accurate gradients but slower updates, while small batches give faster updates but noisier gradients
- ◆ **Optimization Challenges**
- ◆ **Non-Convexity:** neural networks have non-convex loss surfaces, so training can converge to different local optima depending on initialization
- ◆ **Initialization:** zero initialization causes symmetry where all neurons behave identically, so use small random values to break symmetry
- ◆ **Learning Rate:** too small leads to slow convergence, too large causes instability, so typically start large and decay over time
- ◆ **Ill-conditioned Loss:** loss surfaces may be steep in some directions and flat in others, slowing or destabilizing training
- ◆ **Saddle Points:** flat regions with near-zero gradients that are not minima, causing slow progress
- ◆ **Improving Optimization**
- ◆ **Momentum:** uses a running average of gradients to reduce zig-zagging and speed up convergence
- ◆ **Adaptive Learning Rates:** adjusts step size per parameter based on gradient magnitude (e.g., Adagrad, RMSProp)
- ◆ **Adam:** combines momentum and adaptive learning rates, making it the most commonly used optimizer

Sequence Labeling

- ◆ **Structured vs Unstructured Prediction: Structured Prediction:** output $\mathbf{Y} = \{y_1, \dots, y_n\}$ has multiple interdependent components with strong correlations and exponential output space; **Unstructured Prediction:** output $\mathbf{Y}$ consists of a single independent label
- ◆ **Sequence Labeling:** Assigns a label y_i to each token x_i in a sequence and is used in tasks like POS tagging, NER, and information extraction, where sequence order and context are crucial
- ◆ **Modeling Goal: Model $P(\mathbf{Y} | \mathbf{X})$** directly instead of independent predictions to capture dependencies between labels
- ◆ **Baseline: Most Frequent Tag:** Assign each word its most frequent label from training data, achieving high token accuracy (~92%) but low sentence accuracy due to error compounding and ignoring context
- ◆ **POS Tagging Insights:** Word meaning depends on context and tag dependencies, so accurate tagging requires global sentence-level modeling
- ◆ **POS Tag Categories: Closed class:** fixed membership function words; **Open class:** frequently updated content words
- ◆ **Types vs Tokens: Types:** unique words; **Tokens:** actual occurrences of words
- ◆ **Sequence Encoders**
- ◆ **CNNs:** capture local patterns using fixed-size windows with parallel computation but limited long-range context
- ◆ **Simple RNNs:** process sequences recurrently but lack future context, are inefficient, and suffer from vanishing/exploding gradients
- ◆ **Bidirectional RNNs:** incorporate both past and future context by processing sequences in both directions
- ◆ **LSTMs:** use gating mechanisms and memory cells to capture long-range dependencies and address gradient issues
- ◆ **BiLSTM-CNN:** combines character-level CNN features with contextual BiLSTM representations
- ◆ **Log-Linear & Structured Models**
- ◆ **Log-Linear Models:** use feature functions $f(\mathbf{x}, \mathbf{y})$ with score $= \mathbf{v} \cdot f(\mathbf{x}, \mathbf{y})$ and normalize to obtain probabilities
- ◆ **Feature Explosion:** combining many features leads to large, sparse feature spaces that are difficult to train
- ◆ **Independence Assumptions:** simplify computation using assumptions like the Markov property where each tag depends on the previous tag
- ◆ **MEMMs vs CRFs: MEMMs:** locally normalized models that predict each tag separately and suffer from label bias favoring states with fewer transitions; **CRFs:** globally normalized models that score entire sequences before normalization, capturing full dependencies and avoiding label bias
- ◆ **Modern Hybrid Model: BiLSTM-CNN-CRF** combines character-level features, contextual encoding, and structured output modeling for strong performance

- ◆ **Decoding: Viterbi Algorithm:** Uses dynamic programming to efficiently find the most probable tag sequence by keeping best partial paths and backtracking. $\text{pi}[j, s] = \max_{\text{over } s'}$ of $\{\text{pi}[j-1, s'] \times \text{transition_score}\}$ Move forward through the sequence. At the end, backtrack to recover the best sequence. This efficiently finds the highest-scoring tag sequence.

Seq2Seq and Neural Machine Translation (NMT)

- ◆ **Seq2Seq vs Sequence Labeling: Sequence Labeling:** output length equals input length, assigning one label per token **Seq2Seq:** input and output lengths can differ, output space is much larger, and output tokens depend on previously generated tokens
- ◆ **Statistical Machine Translation (SMT):** Uses **word alignment models** where words are aligned based on co-occurrence frequency in translation pairs
- ◆ **Autoregressive Factorization:** Models sequence probability using chain rule: $p(\mathbf{Y}|\mathbf{X}) = \prod p(y_t | \mathbf{y}_{<t}, \mathbf{X})$, where each token depends on all previous outputs and input, with no independence assumptions
- ◆ **Encoder-Decoder Architecture; Encoder:** converts input sequence into a sequence of vectors **Decoder:** generates output tokens one at a time using encoder outputs and previously generated tokens
- ◆ **Training Seq2Seq:** Uses **Maximum Likelihood Estimation (MLE)** to maximize probability of correct sequence: **Teacher forcing** feeds the true previous token during training: Gradients flow through both encoder and decoder, training them jointly
- ◆ **Special Tokens <BOS>: start of sequence <EOS>: end of sequence, used to stop generation**
- ◆ **Decoding Algorithms**
- ◆ **Exhaustive Search:** considers all possible sequences (V^T), computationally infeasible
- ◆ **Greedy Search:** selects best token at each step, fast but may miss optimal sequence
- ◆ **Beam Search:** keeps top K candidate sequences, balances efficiency and performance but not guaranteed optimal
- ◆ **Bottleneck Problem:** Encoder compresses entire input into a single vector, causing information loss and difficulty handling long sequences
- ◆ **Attention Mechanism:** At each decoding step, uses all encoder states instead of a single vector to compute a context vector
- ◆ **Attention Computation**
- ◆ **Compute scores $e_{tj} = \text{sim}(\mathbf{c}_j, \mathbf{h}_t)$** to measure relevance of each input position
- ◆ **Apply softmax** to obtain attention weights α_t
- ◆ **Compute context vector** as weighted sum of encoder outputs
- ◆ **Benefits of Attention:** Solves bottleneck problem, improves handling of long sequences, and reduces information loss
- ◆ **Types of Attention**
- ◆ **Dot-product attention:** uses simple dot product between vectors
- ◆ **Multiplicative attention:** introduces a learnable weight matrix for more flexibility
- ◆ **Additive attention:** combines vectors with learned transformations and non-linearity for higher expressiveness

Pytorch

- ◆ **Tensors:** A tensor is a multi-dimensional array (like NumPy) that can run on CPU or GPU, with metadata including shape, dtype, stride, and device. Created from lists, NumPy arrays, or other tensors using clone/detach methods
- ◆ **Tensor Operations:** Matrix multiplication uses **@**, **torch.matmul()**, **torch.mm()** (2D), **torch.bmm()** (3D), **torch.einsum()**; Useful methods include **.item()** for scalar extraction, **.clamp()** for value limits, and **.t()** for transpose
- ◆ **Autograd:** Automatically computes gradients using the **chain rule**, avoiding manual differentiation; Enabled by **requires_grad=True**, building a dynamic computation graph during forward pass; Calling **loss.backward()** performs backpropagation and then destroys the graph
- ◆ **Computational Graph:** Graph consists of **nodes (tensors)** and **edges (operations)** with **grad_fn** storing backward computation; Gradients accumulate by default, so must be reset before each update
- ◆ **Gradient Handling:** Reset gradients using **model.zero_grad()** or **param.grad.zero_()**; Weight updates should be wrapped in **torch.no_grad()** to avoid tracking updates in the graph and prevent incorrect gradients
- ◆ **Why torch.no_grad():** Prevents PyTorch from computing gradients through weight updates, which would otherwise corrupt the computation graph
- ◆ **Custom Autograd:** Use **torch.autograd.Function** to define custom forward and backward passes when needed
- ◆ **torch.nn Module:** Provides higher-level abstraction for defining models as layers and handling forward/backward passes automatically; Includes built-in layers and loss functions for easier model construction
- ◆ **Optimizers:** Use **torch.optim** for parameter updates with advanced optimizers like Adam; **optimizer.step()** updates all parameters using stored gradients
- ◆ **Dataset & DataLoader:** **Dataset** handles data loading and indexing; **DataLoader** provides batching, shuffling, and efficient data loading; Uses padding and **collate_fn** for variable-length inputs
- ◆ **Training Loop:** Standard process: forward pass → compute loss → zero gradients → backward pass → optimizer step
- ◆ **Prompting & Few-Shot Learning:** **Prompting** reformulates tasks as text generation (e.g., cloze or prefix prompts); **Few-shot learning** provides a small number of examples in the prompt, enabling in-context learning without parameter updates
- ◆ **Parameter-Efficient Fine-Tuning (PEFT):** Goal: adapt models by training **<1% of parameters** instead of full fine-tuning
- ◆ **Methods: Adapters:** small trainable layers inserted into model; **Prefix tuning:** trainable virtual tokens influencing attention; **LoRA:** low-rank updates $\mathbf{W} + \mathbf{A} \times \mathbf{B}$, reducing memory and compute
- ◆ **Alignment Techniques: Instruction Tuning:** fine-tune on human-written instructions to improve task following; **RLHF:** train reward model from human preferences and optimize model outputs accordingly

Distributed Training Large-scale training requires model architecture (e.g., Transformers), massive data, clear objective (next-token prediction), and powerful hardware (many GPUs) Distributed Training Goals: Must satisfy three criteria: minimal redundant computation, minimal peak memory usage, and minimal communication overhead GPU Communication Primitives All-Reduce: sums tensors across GPUs and shares full result with all Reduce-Scatter: sums tensors and distributes partitions across GPUs All-Gather: collects partitions from all GPUs to reconstruct full tensor Relationship: All-Reduce = Reduce-Scatter + All-Gather Distributed Data Parallel (DDP): Each GPU holds a full copy of model parameters and optimizer states and processes a different subset of data; Gradients are synchronized across GPUs using All-Reduce after each backward pass; Issues: high memory cost due to full replication and redundant computation during optimization DDP Optimization Insight: Using Reduce-Scatter + All-Gather can partially reduce redundancy, but full parameters and gradients are still needed during computation Fully Sharded Data Parallel (FSDP): Splits parameters, gradients, and optimizer states across GPUs instead of replicating them; Only gathers parameters for a layer when needed and releases them afterward; Benefits: minimal memory usage, no redundant optimizer state, and efficient communication Model / Tensor Parallelism: Splits computation of a single layer across multiple GPUs when it cannot fit on one GPU; Column parallel: split weight matrix by columns so each GPU computes part of output features; Row parallel: split weight matrix by rows and combine outputs using All-Reduce; Applied to attention by splitting Q, K, V projections and combining outputs across GPUs Constraint of Tensor Parallelism: Requires heavy communication, so typically used within a single node of GPUs Memory Optimization Techniques: Activation checkpointing: recomputes intermediate activations during backward pass to reduce memory usage; Mixed precision training: uses lower precision formats to reduce memory and improve efficiency Precision Types: FP32: high precision, large memory usage, wide numerical range; FP16: lower precision, smaller range, risk of overflow; BF16: similar range to FP32 with reduced precision, preferred for stable large-scale training Pretraining Train large models on massive unlabeled text with general objectives, then fine-tune for downstream tasks because unlabeled data is abundant while labeled data is expensive. Pretraining learns general language knowledge (syntax, semantics, reasoning) and fine-tuning adapts it to specific tasks Contextualized Embeddings: Word meaning depends on context, so models must produce context-dependent representations instead of fixed vectors; Classic embeddings (Word2Vec, GloVe) assign one vector per word, failing to capture polysemy and context Transformers Advantage: Use self-attention to model full context, allowing each word representation to depend on all other words Pretraining Architectures: Encoder Models (BERT) Learn bidirectional contextual representations using full left and right context; Masked Language Modeling (MLM) masks ~15% of tokens and predicts them using context; Uses softmax + cross-entropy loss over vocabulary; Input representation combines token, segment, and position embeddings (sequence length typically 512 tokens); Strength: excellent for understanding tasks; Weakness: not suitable for generation Decoder Models (GPT): Use autoregressive language modeling (next-token prediction) with left-to-right context only; Strength: strong for text generation; Limitation: cannot use future context Encoder-Decoder Models (BART / T5): Combine understanding and generation using denoising autoencoding, where encoder reads corrupted input and decoder reconstructs original text; Noise strategies include masking, deletion, text infilling, sentence permutation, and document rotation; Strength: strong for summarization, translation, and generation tasks Tokenization (BPE): Splits text into subword units using iterative merging of frequent character pairs; Uses longest-match tokenization at inference Pros: eliminates unknown words, captures subword structure, balances word- and character-level representations; Cons: captures shallow similarity and may over-segment morphologically rich languages Fine-Tuning: Add a task-specific head and train on labeled data, updating all parameters; Requires small learning rate and strong regularization to avoid overwriting pretrained knowledge Prompting & In-Context Learning: Reformulate tasks as text generation using prompts instead of retraining; Pipeline: create prompt template → generate output → map output to label; Types: cloze prompts (fill-in-the-blank) and prefix prompts (continuation); Few-shot learning uses <20 examples inside prompt for in-context learning Parameter-Efficient Fine-Tuning (PEFT): Avoid full fine-tuning by updating <1% of parameters while freezing base model Methods: Adapters: small trainable layers inserted into Transformer Prefix tuning: learn virtual tokens influencing attention LoRA: low-rank weight updates $W + A \times B$ for efficient adaptation; Benefit: similar performance to full fine-tuning with much lower compute and storage Alignment Techniques: Pretrained models are not aligned with human intent by default Instruction tuning trains models on human-written instructions for better task following RLHF uses human preference rankings to train a reward model and optimize outputs for helpfulness and safety Posttraining Pre-training: Objective: maximize $P(\text{sequence})$ via MLE; Chain rule: $\log P(w_1) + \log P(w_2 w_1) + \dots$; Learns: general knowledge, language patterns;	Acts as: autocomplete system; Data: high quantity, low quality; imitates average internet text Post-training: Goal: control behavior, alignment, quality; Methods: Fine-tuning (SFT); Reinforcement Learning (RL) Supervised Fine-Tuning (SFT): Dataset: (prompt, response) pairs; Objective: maximize $P(\text{response} \text{prompt})$; Loss: computed only on response tokens; Effect: selects desirable behaviors from pre-training “Distant” Supervision: Use: (prompt, final answer) instead of full response; Advantage: easier data collection Why Fine-Tuning Alone Fails: 1. Data Scale - High-quality data scarce, expensive; 2. Exposure Bias: Training: human-written prefixes; Testing: model-generated prefixes; Problem: model never sees its own errors; 3. Factuality / Guessing: Encourages: hallucination; Model forced to: output something even if unsure Key Insight: SFT = imitation learning; Known to be: suboptimal Reinforcement Learning (RL) Motivation: Optimize: entire output quality; RL Basics: Agent: model; State: prompt + generated tokens; Action: next token; Reward: usually at end of sequence Language Model as Policy: Policy: $P(\text{next token} \text{context})$; Objective: maximize expected reward Policy Gradient: Goal: maximize expected reward via gradient ascent Key trick: $V_p = p \cdot V \log p$; Final form: $E[V_0 \log p_\theta(y x) - R(x,y)]$ Algorithm (Recipe): 1. Sample k rollouts 2. Compute rewards 3. Estimate gradient (average) 4. Update θ 5. Repeat RL with Verifiable Rewards (RLVR) Example: math problems Reward: 1 (correct), 0 (wrong); Fully automatic evaluation RLHF (Reinforcement Learning from Human Feedback) 3-Step Pipeline: 1. SFT (imitation learning) 2. Reward model training 3. RL optimization (e.g., PPO) RLHF Data: Prompts: annotators + real users Demonstrations: ideal responses Rankings: human preference ordering Annotators: ~40 contractors (InstructGPT) Source: Upwork, ScaleAI; Require: screening tests Reward Model: Separate pretrained LM; Fine-tuned to predict: human preferences Training Setup: Input: prompt x, outputs yw (win), yl (lose); Output: scalar $r(x, y)$ Objective: Ensure: $r(x, yw) > r(x, yl)$; Loss: logistic regression; Interpreted as: binary classification RL Optimization (RLHF Step 3): Use: policy gradient; Goal: maximize expected reward RLHF & Factuality Encourages: correct answers when known, “I don’t know” when uncertain; Reduces: hallucination Advantage (Core RL Concept) Problem: raw rewards → high variance; Solution: Advantage A = relative performance Behavior: $A > 0 \rightarrow$ increase probability; $A < 0 \rightarrow$ decrease probability GRPO vs PPO: GRPO: Advantage: $(R_i - \text{mean}) / \text{stdev}(\text{z-score})$; No extra model; Needs: many rollouts PPO: Uses: critic (value model); Advantage: fewer rollouts needed; Tradeoff: extra model required PPO = Actor-Critic; Actor = LLM; Critic = value estimator On-Policy vs Off-Policy; On-policy: sampling = current policy Off-policy: samples from θ_{old}; Why off-policy? Sampling is: expensive; Want: multiple updates per batch Importance Sampling: Correct off-policy mismatch: $\text{weight} = P_\theta(y x) / P_{\theta_{\text{old}}}(y x)$ PPO Objective: Maximize: importance-weighted advantage Clipping (PPO Stabilization): Limit ratio: $[1 - \epsilon, 1 + \epsilon]$ Behavior: $A > 0$: cap at $1 + \epsilon$; $A < 0$: floor at $1 - \epsilon$ Effect: prevents: large policy drift; beyond clip: no gradient DPO (Direct Preference Optimization): Alternative to RLHF Idea: combine: reward model + RL → single step; Directly trains on: preference data Optimal Policy Insight: Optimal policy: proportional to: $\exp(\text{reward}) \times \text{old policy}$; Includes: KL regularization (β) DPO Loss: Uses: preference pairs (yw, yl); Form: logistic loss over: log probability ratios; No: rollouts required DPO vs PPO / GRPO Sequential DPO: iterative relabeling + retraining	Evaluation uses reference-based comparison, where model output is compared to human-written references BLEU (Precision-Based): Measures n-gram precision ($n=1-4$) comparing prediction to reference Formula: $\text{BLEU} = \text{BP} \cdot (p_1 \cdot p_2 \cdot p_3 \cdot p_4)^{1/4}$ pn: fraction of predicted n-grams appearing in reference Brevity Penalty (BP): equals 1 if prediction length $\geq$ reference, otherwise $e^{1 - \text{ref_len}/\text{pred_len}}$ to penalize short outputs Multiple references: an n-gram is correct if it appears in any reference ROUGE (Recall-Based): Measures n-gram recall, commonly used for summarization Key limitation: BLEU focuses on precision, ROUGE focuses on recall Limitations of Word Overlap: High word overlap does not guarantee correct meaning, e.g., “took class” vs “taught class” yields high BLEU but incorrect semantics Model-Based Metrics: BERTScore: uses contextual embeddings and cosine similarity to match meaning rather than exact words; computes precision, recall, and F1 via token similarity MiniCheck: supervised classifier predicting whether a claim is supported by a document, trained using synthetic data with hard negatives LLM-as-a-Judge: Uses an LLM to evaluate outputs directly, enabling reference-free and customizable evaluation, but introduces potential biases Factuality Metric Pitfalls: Inconsistency: different metrics disagree on which outputs are incorrect; TPR vs TNR trade-off: balanced datasets hide poor hallucination detection (low TNR); System-level error: aggregated scores can over- or under-estimate true performance; Misleading headroom: metrics can incorrectly estimate room for improvement Surface-Level Similarity Bias: Metrics assume high overlap implies factual correctness; Low overlap leads to penalizing paraphrases (low TPR), while high overlap fails to detect subtle errors (low TNR) Calibration Fix: Tune metric thresholds per dataset using a small labeled set to reduce system-level bias; optimizing balanced accuracy alone can worsen bias Diversity Metrics: Important for open-ended tasks and synthetic data generation, as RL training can cause mode collapse; Self-BLEU: compute BLEU across multiple outputs for the same prompt; lower score indicates higher diversity Dataset Design Principles: Important factors: task, domain, ecological validity, style/length, difficulty, noise, size; Key insight: reported accuracy is meaningless without specifying the dataset Dataset Sourcing: Found Data: naturally occurring labeled data (e.g., IMDb reviews with ratings); Created Data: generated by crowdworkers, experts, or LLMs Stress Testing: Used to identify model weaknesses and ensure reliable deployment Adversarial Examples: Designed to exploit model shortcuts, such as overreliance on word overlap in QA Adversarial Data Collection: Humans interact with models to continuously discover failure cases (e.g., Dynabench-style evaluation) Long-Tail / Unlikely Examples: Models often fail on rare or unlikely tasks, even if they perform well on common patterns Safety Evaluation: Measures whether models refuse harmful or malicious requests, including resistance to jailbreak prompts Multimodal Streetlight Effect: researchers focus on easy-to-analyze data (text) instead of where true understanding lies Image Representation: Images represented as 3D tensors ($W \times H \times 3$) where each pixel contains RGB color values Fully-Connected (FC) Models for Images: Treat images as flattened vectors, which requires millions of parameters and ignores spatial relationships, making them inefficient Vision Transformers (ViT): Process: split image into patches, flatten them, add position embeddings, and pass through a Transformer encoder to capture global relationships ViT Pre-training: Uses masking or corruption of patches and trains the model to predict missing content, similar to BERT-style learning Multimodal Architectures CLIP (Late Fusion): Uses separate image and text encoders that map both into a shared embedding space, trained by pulling matching pairs together and pushing non-matching pairs apart ViLT (Early Fusion): Combines text tokens and image patches into a single Transformer, allowing direct modeling of cross-modal interactions Generative Vision-Language Models: Architecture: vision encoder feeding into an LLM decoder, enabling generation of text responses about images and videos (e.g., Qwen3-VL) Vision-Language-Action (VLA): Extends multimodal models to produce physical actions instead of just text Embodied AI: Tasks include Embodied Question Answering and vision-language navigation, where the agent must interact with and move through an environment Action Tokens: Models output structured action commands (e.g., robot coordinates or gripper movements), enabling physical tasks like object manipulation (e.g., OpenVLA, MolmoAct) Speech Processing (Speech LLMs / SLMs): Speech is converted into acoustic tokens using audio tokenizers (codecs like Mimi), which are then processed by Transformer models Multimodal ASR (Speech + Vision): Combines audio and visual context, allowing models to improve recognition when audio is unclear (e.g., visual context helps interpret muffled words) Signed Language Processing: Uses video input processed into pose representations and modeled with Graph Neural Networks, capturing handshake, motion, and spatial location without relying on text
--	---	--

Limitations: knowledge cutoff prevents answering recent events, long-tail knowledge is poorly represented, private/internal data is inaccessible, and very long documents are hard to handle ◆ Information Retrieval (IR): Input: query + document collection; Output: ranked list of relevant documents; Goal: retrieve relevant documents efficiently and accurately ◆ IR Challenges: Accuracy: must find relevant documents among millions of irrelevant ones; Latency: must return results quickly despite massive datasets ◆ Vector Space Model: Represent queries and documents as vectors, where relevance is computed using vector similarity, enabling both accuracy and efficient retrieval ◆ Sparse Retrieval (Traditional): Sparse Vectors: Represent text as high-dimensional vectors with many zeros, where each dimension corresponds to a vocabulary term ◆ Cosine Similarity: Measures similarity between vectors using normalized dot product, and computation is efficient because only query terms contribute ◆ TF-IDF: Combines TF (log-scaled frequency to avoid dominance of repeated words) and IDF (log(N/df) to emphasize rare words), so rare terms are weighted more heavily while common words are down-weighted ◆ Inverted Index: Maps each term to a posting list of document IDs, allowing the system to skip all documents without query terms and compute similarity only for relevant candidates ◆ Inverted Index Optimizations: Improves efficiency by focusing on high-IDF terms, intersecting posting lists, using sorted lists, and maintaining champion lists of top documents per term ◆ Query Expansion: Expands queries with synonyms or related words, helping retrieve relevant documents that do not share exact terms ◆ Sparse Retrieval Summary: Fast, memory-efficient, and requires no training, but limited because it relies on exact word overlap and misses semantic similarity ◆ Dense Retrieval (Neural): Dense Vectors: Use neural models like BERT to encode text into dense vectors capturing semantic meaning ◆ Dense Passage Retrieval (DPR): Uses two BERT encoders for queries and passages, taking the [CLS] vector and computing similarity via dot product ◆ DPR Training: Trains using query, positive passage, and negative passages, optimizing a softmax loss to distinguish relevant from irrelevant passages ◆ Training Data Construction: Uses BM25-retrieved passages containing answers as positives, hard negatives without answers, and in-batch negatives for efficiency ◆ Document Chunking: Splits long documents into ~100-word chunks, ensuring each chunk can be effectively encoded and retrieved ◆ Dense Retrieval Efficiency (IVF): Partitions vector space into clusters (cells) using k-means, allowing search to focus only on nearest clusters rather than all documents ◆ Product Quantization (PQ): Compresses vectors by splitting into sub-vectors and quantizing them, reducing memory usage by up to 32x ◆ Dense Retrieval Summary: Captures semantic similarity better than sparse methods, but has higher memory, computation, and setup costs, and single vectors may not fully represent long documents ◆ ColBERT (Advanced Neural IR): Uses one vector per token instead of one per document, enabling finer-grained representation ◆ Scoring (MaxSim): Computes similarity by matching each query token to its best matching document token and summing the scores ◆ Late Interaction: Encodes query and document separately and compares representations at the end, unlike early fusion which combines them earlier but is inefficient for retrieval ◆ Retrieval Strategy: Stores token vectors in an IVFPQ index and uses approximate scoring to prune candidates before full scoring, improving efficiency ◆ Trade-offs: Provides fine-grained semantic matching, but requires significantly more memory and is slower than simpler methods ◆ Re-ranking Pipeline: Uses a two-stage approach where fast methods like BM25 retrieve top candidates and expensive models like BERT or ColBERT re-rank them, balancing efficiency and accuracy ◆ Evaluating IR: Precision@K: fraction of top-K retrieved documents that are relevant; Recall@K: fraction of all relevant documents retrieved in top-K; MRR: average of inverse rank of first relevant document, rewarding higher-ranked correct results ◆ Retrieval Augmented Generation (RAG): Pipeline: retrieve top-K documents, provide them as context, and generate answer with LLM, enabling grounded responses ◆ Why RAG is Needed: Addresses knowledge cutoff, long-tail knowledge, private/internal data, and long-document limitations ◆ Multi-stage RAG: Iteratively performs retrieval and reasoning, refining queries until sufficient information is found ◆ IRCoT (Interleaved Retrieval + CoT): Alternates retrieval steps with reasoning steps, enabling multi-hop reasoning across documents ◆ Deep Research Systems: LLMs interleave natural language reasoning with search actions, dynamically retrieving and evaluating information	◆ Attention Visualization: Visualizes which tokens attend to others, revealing patterns like pronoun → antecedent relationships ◆ Linear Probes: Train a simple linear classifier on frozen hidden states to predict concepts; if successful, the concept is linearly represented Example: sentiment and truthfulness are linearly separable in hidden states Limitation: requires labeled data and predefined concepts ◆ Residual Stream: Transformer hidden state is a residual stream that accumulates outputs from all attention and MLP layers, so later representations contain information from earlier ones ◆ Logit Lens (Vocabulary Projection): Apply unembedding to intermediate hidden states or module outputs to inspect which tokens the model is predicting at different layers ◆ Interpreting MLPs: MLPs act as key-value systems where the first layer detects concepts and the second layer outputs corresponding value vectors, and these value vectors map to coherent word clusters when unembedded ◆ Superposition: Hidden states encode many concepts in overlapping linear directions, meaning more concepts exist than dimensions, so representations are entangled ◆ Sparse Autoencoders (SAEs): Learn a dictionary of features larger than hidden dimension and represent hidden states as sparse combinations of these features, enabling disentangling of concepts. Interpretation: identify where features activate and infer human-readable concepts ◆ Causal Methods (Write/Intervention) ◆ Loss-of-Function (Ablation): Remove a component and observe if behavior disappears, showing the component was necessary for that behavior ◆ Gain-of-Function (Rescue / Patching): Add or restore a component in a system lacking it and observe if behavior appears, showing the component was sufficient ◆ Steering: Add a steering vector (difference between concept vs non-concept states, probe weights, or SAE direction) to the residual stream during generation to influence behavior Controlled by scaling factor α, where too large values disrupt normal behavior Works best in middle layers, since early layers lack concepts and late layers cannot influence downstream computation ◆ Clamping: Instead of shifting representations, directly set a specific SAE feature to a fixed value, enabling precise control over concepts ◆ Activation Patching (Causal Tracing): Replace activations from a correct run into a corrupted run to see which components restore correct behavior, identifying causal roles ◆ Knowledge Localization: Experiments show: MLP layers extract factual knowledge (e.g., Space Needle → Seattle); Attention layers move information to final token for prediction ◆ Retrieval Heads: Certain attention heads specialize in retrieving information from long contexts, and ablating them significantly reduces performance, proving they are necessary Agents Traditional LLMs only generate text, but many tasks require real-world actions like retrieving data, computing values, or editing files. Examples: answering real-time questions, performing calculations, scheduling, modifying code ◆ Core Idea: LLMs can take actions via tools to help generate final answers, extending beyond static knowledge ◆ Semantic Parsing (Precursor): Converts natural language into a formal program, which is executed by an external system to produce the answer ◆ Tool Use Loop: Process: LLM generates tokens → emits special tool-call sequence → tool executes → result inserted into context → LLM continues generation ◆ Types of Tools: Knowledge access: search engines, databases; Computation: calculators, Python interpreters; World interaction: APIs for weather, calendars, etc. ◆ Toolformer (Formatting + Training): Uses special tokens like “[”, “]”, “->” to mark tool calls, and after “->” the tool is executed and result appended; Trained via supervised data with API calls using standard LM loss ◆ ToolkenGPT: Introduces special “toolken” tokens for each tool, where generating a toolken triggers a separate tool mode; Training keeps base LM frozen and only updates toolken embeddings, supervising both tool selection and usage ◆ Tool Usage (Prompting): LLM first selects tool in reasoning mode, then switches to tool mode to generate arguments using demonstrations and context ◆ Standard Protocols: OpenAI Function Calling: tools defined via JSON schema, model outputs {name, arguments}; MCP (Model Context Protocol): unified interface where server lists tools and executes calls, simplifying multi-tool integration ◆ What is an Agent? System that takes multiple iterative actions in an environment using tools to achieve a goal ◆ Types of Agents: Deep Research Agents: perform search and browsing to synthesize information; Coding Agents: edit files, run code, debug, and manage repositories; Web Agents: interact with websites via multi-step actions; Embodied Agents: act in physical or simulated environments ◆ Agent Design Considerations ◆ Representation (Environment Input) Text: default representation for many tasks like research or simulated environments Images: used for navigation or perception but struggle when containing dense text or charts Web Pages: represented via HTML, screenshots, or structured views combining multiple modalities Set of Marks: augment images with labeled regions so agents can refer to objects via IDs Codebases: shown partially (e.g., 100-line window), requiring navigation actions to explore large repositories ◆ Action Space: Must include both: environment understanding actions (searching, browsing, exploring context) and task execution actions (editing files, clicking elements, running code) Example (coding agents): actions include file editing, searching, scrolling, running scripts, and querying documentation	◆ Search Strategy Insight: Summarized search works best by showing all results at once and retrying if too many results are returned ◆ Web Agent Actions: Use browser-like interfaces, selecting elements via coordinates or IDs and interacting programmatically ◆ Planning in Agents ◆ Top-Down Planning: Generate full plan first, then execute step-by-step, but fails when unexpected errors occur ◆ ReAct (Interleaving): Interleave Thought → Action → Observation, allowing plans to adapt dynamically based on feedback ◆ Reflexion: Agent self-evaluates progress, identifies errors, and retries using reflections, improving recovery from failures ◆ Reasoning Models: Models like DeepSeek-R1 use reinforcement learning to re-evaluate, backtrack, and try alternative strategies during reasoning Risks ◆ Fairness & Bias Issues ◆ Word Vector Bias: Embeddings show strong correlation between real-world gender workforce gaps and embedding similarity to male/female terms, meaning societal bias is encoded mathematically ◆ Generative Bias: Models learn biases from training data, leading to stereotypical outputs in text and image generation (e.g., men as programmers, women as housekeepers) ◆ Failed Bias Mitigation: Heavy-handed fixes like injecting diversity keywords can produce unrealistic or historically incorrect outputs, showing naive debiasing can backfire ◆ Real-World Impact: AI systems have caused harm, e.g., Amazon recruiting tool penalized resumes containing “women’s” and favored male candidates, demonstrating real deployment risks ◆ Bias in Retrieval / Screening: Models show gender and racial bias when ranking identical resumes differing only by name, favoring male and white-coded names ◆ Language & Tokenization Bias: Tokenization favors English, causing a “tokenization premium” where rare languages require more tokens, increasing cost and reducing accuracy ◆ Bias Under Uncertainty: Models default to most common or stereotypical answers when uncertain, e.g., predicting major cities or developed regions ◆ Memorization & Data Privacy ◆ Extractable Memorization: Models can reproduce exact training data suffixes given prefixes, especially when data appears frequently ◆ Memorization Trends: Memorization increases with more repetitions in training data and larger model size, but rarely occurs for single occurrences ◆ Dilution Effect: Training on larger corpora reduces memorization because relative frequency of repeated data decreases, lowering leakage risk ◆ Test Set Contamination: Models perform better on pre-cutoff benchmarks, indicating they have seen answers during training rather than reasoning ◆ Copyright Concerns: Models can generate verbatim copyrighted text, leading to lawsuits such as the New York Times case ◆ Data Curation: Pipelines remove up to ~98% of raw data using filtering and deduplication, showing importance of dataset quality ◆ Detecting LLM-Generated Text ◆ Watermarking: Idea: bias generation toward certain “green” tokens ◆ Hard Watermark: Restrict output to green tokens, but hurts fluency significantly ◆ Soft Watermark: Add small bias δ to green tokens, creating subtle statistical signature detectable via z-test against expected 50/50 distribution ◆ Deployment Challenge: Not widely used because users dislike detectable outputs, reducing adoption ◆ Risks: LLMs enable scams, phishing, and large-scale social media manipulation ◆ Sycophancy: Models tend to agree with users and validate beliefs, especially due to RLHF optimizing for user satisfaction ◆ Medical Risks: Models often fail to correct false assumptions, e.g., incorrect medical beliefs are not challenged ◆ Psychological Harm: Case studies show models can reinforce harmful behaviors, including extreme scenarios like validating suicidal intent ◆ High-Impact Threats: LLMs may assist in bioweapons development, cyberattacks, or other large-scale harm ◆ Measuring Risk (Uplift Testing): Compare task success with vs without LLM assistance to assess real-world impact ◆ Jailbreaking: Modifying prompts to bypass safety guardrails. Find suffix that maximizes probability of unsafe response, enabling systematic bypass of safety mechanisms ◆ Attack Types: Closed models: limited to prompt-based attacks; Open-weight models: allow fine-tuning, steering vectors, or weight modification, making attacks more powerful ◆ Emergent Misalignment: Fine-tuning for one harmful behavior can cause unexpected harmful behaviors elsewhere, showing linked representations ◆ Risks with Agentic Systems ◆ Autonomous Risks: Agents can take unintended actions like deleting emails, making purchases, or leaking private data ◆ Off-Script Behavior: Agents may ignore instructions and act unpredictably, including generating harmful content or defaming users ◆ Security & Liability: Raises questions about responsibility when agents cause harm or misinformation An LSTM uses gates to control memory flow: the forget gate $f_t = \sigma(W_{ff}[h_{t-1}, x_t] + b_f)$ decides what to discard from the cell state c_{t-1}, the input gate $i_t = \sigma(W_{if}[h_{t-1}, x_t] + b_i)$ and candidate $\tilde{c}_t = \tanh(W_{c}[h_{t-1}, x_t] + b_c)$ decide what new information to add, updating memory as $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, and the output gate $o_t = \sigma(W_{of}[h_{t-1}, x_t] + b_o)$ produces the hidden state $h_t = o_t \odot \tanh(c_t)$; here σ is sigmoid and $\odot$ is element-wise multiplication.
--	---	--

TRANSFORMERS I & II + ADVANCED TRANSFORMERS	RESIDUAL CONNECTIONS Formula: $Y = f(X) + X$ Purpose: - Enables gradient flow in deep networks - Allows identity mapping - Prevents degradation Used around: - Attention layer - FFN layer	EFFICIENT ATTENTION METHODS Sparse / Sliding Window Attention: - Attend to nearby tokens only - Pros: efficient - Cons: loses global context Chunk-wise Attention: - Process chunks independently - Problem: breaks long-range dependencies Linear Attention: - Rewrites computation: $(QK^T)V \rightarrow Q(K^TV)$ - Reduces complexity from $O(n^2)$ to $O(n)$ - Uses kernel approximations Low-Rank Approximation: Linformer: - Projects sequence into smaller dimension Luna: - Uses pack/unpack compression Trade-off: - Faster, less memory - Possible information loss
MOTIVATION: FROM RNNs TO TRANSFORMERS NLP evolution: Statistical models $\rightarrow$ RNN $\rightarrow$ LSTM $\rightarrow$ Transformers $\rightarrow$ LLMs RNN limitations: - Sequential computation (slow, not parallelizable) - Vanishing/exploding gradients - Single vector bottleneck (compress entire sequence into one state) - Poor long-range dependency modeling Transformers: - Remove recurrence entirely - Use attention-only mechanism - Fully parallelizable "Attention is All You Need"	NORMALIZATION Batch Normalization: - Across batch dimension Problems: - Unstable for small batches - Train-test mismatch Layer Normalization (standard): - Across feature dimension per token - Works for variable-length sequences - Stable for NLP RMSNorm (modern): - No mean subtraction - Faster and more stable - Used in LLaMA	
CORE MECHANISM: SELF-ATTENTION Definition: Computes a weighted combination of all tokens in a sequence to produce contextualized representations. Q, K, V projections (very important): $q_i = W_Q x_i$ $k_i = W_K x_i$ $v_i = W_V x_i$ Attention formula: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$ Scaling by $\sqrt{d}$ (critical intuition): - Dot products grow large with dimension - Softmax becomes extremely peaked - Leads to tiny gradients - Scaling stabilizes training Interpretation: - Query attends to keys to retrieve values - Produces context-aware representation - Maps variable-length input $\rightarrow$ fixed-size vector per token	PRE-NORM VS POST-NORM Post-Norm (original): $Y = \text{LayerNorm}(f(X) + X)$ - Can cause unstable gradients in deep models Pre-Norm (modern): $Y = f(\text{LayerNorm}(X)) + X$ - More stable for deep networks - Slight theoretical trade-offs	LONG RANGE ARENA (LRA) Benchmark for long-context models Measures: - Accuracy - Speed - Memory efficiency
ADVANTAGES OF SELF-ATTENTION - Captures long-range dependencies in one step - Fully parallelizable (no sequential dependency) - Avoids vanishing gradients - Produces contextual embeddings	POSITIONAL ENCODING Problem: - Self-attention is order-invariant Solution: $x_i = x_i + p_i$ Sinusoidal encoding: - Uses sine and cosine functions - Encodes relative position - Generalizes to longer sequences RoPE (Rotary Positional Embedding): - Rotates Q and K vectors - Encodes relative position directly - Used in modern LLMs	ADVANCED TRANSFORMER VARIANTS Mega: - Replaces attention with: - Exponential Moving Average (EMA) - Gated attention - Linear complexity - Good long-sequence performance Megalodon: - Improved Mega - Uses complex EMA + timestep normalization - More data-efficient Gecko: - Focus: length generalization - Uses sliding chunk attention + adaptive memory - Maintains performance on longer sequences
MULTI-HEAD ATTENTION - Split Q, K, V into multiple heads - Each head learns different relationships Per head: $\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$ Final output: $\text{concat}(\text{head}_1, \dots, \text{head}_h) W_O$ Benefits: - Higher expressiveness - Captures multiple interaction patterns	TRANSFORMER VS RNN RNN: - Sequential $O(n)$ complexity - Weak long-range memory Transformer: - Parallel $O(n^2 d)$ time, $O(n^2)$ memory - Strong global context	EFFICIENCY IMPROVEMENTS (TRANSFORMER-II) SwiGLU: - Replaces ReLU/GELU in FFN - Better performance MQA (Multi-Query Attention): - One shared K/V for all heads - Faster, less memory - Slight accuracy drop GQA (Grouped Query Attention): - Groups share K/V - Best trade-off between speed and quality
MASKED AND CROSS ATTENTION Masked attention (decoder): - Future tokens set to -infinity before softmax - Prevents information leakage - Ensures autoregressive generation Cross-attention: - Query = decoder representation - Key, Value = encoder outputs - Allows decoder to attend to input sequence	TRANSFORMER LIMITATIONS Quadratic complexity: - Time: $O(n^2 d)$ - Memory: $O(n^2)$ - Expensive for long sequences Weak inductive bias: - No built-in locality (CNN) - No sequential bias (RNN) Length generalization problem: - Performs poorly on longer sequences than training - Caused by positional encoding + weak bias	MIXTURE OF EXPERTS (MoE) Core idea: - Sparse model - Only subset of parameters used per token Router (gate): - Computes scores for experts Top-K routing: - Select top K experts - Output = weighted combination Benefits: - Huge parameter count - Low compute per token
TRANSFORMER ENCODER BLOCK Repeated N times Each block contains: - Multi-head self-attention - Feed-forward network (FFN) - Residual connections - Layer normalization	LONG CONTEXT CHALLENGES - KV cache grows with sequence length - Memory usage increases linearly during inference - High GPU memory cost - Communication overhead in distributed systems KV cache: - Stores past keys and values - Avoids recomputation - Becomes major bottleneck	MOE CHALLENGES (HIGH-YIELD) Representation collapse: - Few experts dominate Load balancing: - Requires auxiliary loss Token dropping: - Limits expert capacity Training instability Communication overhead: - Requires All-to-All communication - Tokens routed across GPUs - Major scaling bottleneck
FEED-FORWARD NETWORK (VERY IMPORTANT) Definition: $\text{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2$ Key properties: - Applied independently to each token (position-wise) - Adds non-linearity - Contains $\sim 2/3$ of total model parameters - Acts as memory / pattern storage Modern variant: - SwiGLU activation replaces ReLU/GELU - Improves performance (used in LLaMA, Mistral)		